

Procesamiento Paralelo de Imágenes mediante

Descomposición por Dominio

Explicación Detallada de las Soluciones y Conclusiones

Ejercicio 1 – Descomposición por Dominio (Datos)

20 de septiembre de 2026

1. Introducción

El presente informe documenta el desarrollo del Ejercicio 1, cuyo objetivo es aplicar la técnica de **descomposición por dominio** para paralelizar el procesamiento de un conjunto de imágenes, específicamente su conversión a escala de grises. Se presenta el planteamiento del problema, la solución secuencial de referencia, la solución paralela implementada, el análisis comparativo entre ambas, y las conclusiones obtenidas a partir de los resultados.

1.1. Planteamiento del problema

Se dispone de un directorio con múltiples imágenes. Se requiere aplicar a cada una la misma operación (conversión a escala de grises) y almacenar el resultado. El reto es determinar si dicha tarea puede acelerarse distribuyendo el trabajo entre varios procesos, en lugar de procesarlas una por una.

2. Solución: Procesamiento Paralelo por Descomposición de Dominio

2.1. Descripción

La solución paralela utiliza el módulo `multiprocessing` de Python, en particular la clase `Pool`, para distribuir la lista de imágenes entre varios procesos que se ejecutan de forma concurrente.

Listing 1: Función de conversión (retorna en lugar de imprimir)

```
1 def convertir_a_gris(ruta_imagen):
2     """Convierte una imagen a escala de grises."""
3     try:
4         imagen = Image.open(ruta_imagen)
5         imagen_gris = imagen.convert('L')
6         nombre_archivo, extension = os.path.splitext(ruta_imagen)
7         ruta_gris = nombre_archivo + "_gris" + extension
8         imagen_gris.save(ruta_gris)
9         return f"Imagen convertida: {ruta_imagen} -> {ruta_gris}"
10    except FileNotFoundError:
11        return f"Error: No se encontro la imagen {ruta_imagen}"
12    except Exception as e:
13        return f"Error al procesar {ruta_imagen}: {e}"
```

Listing 2: Distribución del dominio de datos entre procesos

```
1 def procesar_imagenes_paralelo(lista_imagenes, num_procesos=None):
2     if num_procesos is None:
3         num_procesos = cpu_count()
4
5     with Pool(processes=num_procesos) as pool:
6         resultados = pool.map(convertir_a_gris, lista_imagenes)
7
8     for resultado in resultados:
9         print(resultado)
```

2.2. Explicación de los componentes clave

- `cpu_count()`: determina cuántos núcleos tiene disponibles el procesador, utilizado como número de procesos por defecto en el *pool*.
- `Pool(processes=num_procesos)`: crea un grupo de procesos trabajadores independientes.
- `with ... as pool:` asegura que los procesos se cierren correctamente al terminar, incluso si ocurre una excepción.
- `pool.map(convertir_a_gris, lista_imagenes)`: reparte automáticamente los elementos de `lista_imagenes` entre los procesos disponibles y ejecuta `convertir_a_gris` sobre cada uno de forma concurrente. Este es el punto exacto donde ocurre la descomposición por dominio: los **datos** se dividen entre los **procesos**, no el algoritmo.
- **Retorno en lugar de print directo**: dentro de un proceso hijo, imprimir directamente podría entremezclar la salida de distintos procesos en la consola; por ello cada proceso retorna un mensaje y la impresión final se realiza de forma centralizada en el proceso principal.
- `if __name__ == '__main__':`: obligatorio al usar `multiprocessing`, ya que cada proceso hijo reimporta el script; sin esta protección se generaría una ejecución recursiva no controlada, especialmente en Windows.

3. Conclusiones

1. La descomposición por dominio resultó una estrategia adecuada para este problema, ya que la conversión de cada imagen es una operación completamente independiente de las demás, sin necesidad de comunicación ni sincronización entre procesos.
2. El uso de `multiprocessing.Pool` junto con `pool.map()` permitió repartir automáticamente el dominio de datos (la lista de imágenes) entre los procesos disponibles, sin que fuera necesario implementar manualmente la lógica de división de la lista.
3. El beneficio real del paralelismo depende directamente de la relación entre el costo de procesamiento por imagen y el overhead de crear y coordinar los procesos: a mayor volumen o complejidad de trabajo, mayor es la ventaja de la versión paralela frente a la secuencial.
4. La estructura `if __name__ == '__main__':` y el retorno de mensajes (en lugar de imprimir directamente dentro de cada proceso) resultaron necesarios para garantizar un comportamiento correcto y una salida ordenada del programa.

5. En general, se concluye que la descomposición por dominio es una técnica efectiva y de baja complejidad de implementación para acelerar tareas de procesamiento masivo de datos independientes, como el procesamiento por lotes de imágenes.